Artificial Intelligence in Software Engineering: Perspectives and Challenges

Kirti Bhandari
Computer Science & Engineering
Dr. B.R Ambedkar National Institute of Technology
Jalandhar, India
kirtib.cs.18@nitj.ac.in

Kuldeep Kumar
Computer Engineering
National Institute of Technology
Kurukshetra, India
kuldeepkumar@nitkkr.ac.in

Amrit Lal Sangal
Computer Science & Engineering
Dr. B.R Ambedkar National Institute of Technology
Jalandhar, India
sangalal@nitj.ac.in

2023 Third International Conference on Secure Cyber Computing and Communication (ICSCCC) | 979-8-3503-0071-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICSCCC58608.2023.10176436

***Abstract*—Artificial Intelligence in Software Engineering is the hottest trend in the rapidly growing software world. AI techniques are powerful and easy to use as it can be easily deployed as key components of the system. The fusion of both these techniques emerges four new innovative research areas which require extensive research in the future. The application of AI techniques in the Software Engineering field poses many opportunities and risks for software organizations and handling these issues is essential before the selection of appropriate techniques. In this paper, we present an introduction, new emerging fields with the fusion of Artificial Intelligence with Software Engineering, challenges, and a conclusion.**

Keywords— Software Engineering, Artificial Intelligence, Machine Learning

I. INTRODUCTION

With the increasing demand for software in every field, the software engineering (SE) field is gaining importance day by day. According to the U.S. Bureau of Labor, carrier opportunities are also increasing for software engineers in the upcoming years. There will be an increment of 22% in the number of jobs in software development in 2030 irrespective of 2020 which is higher than the average growth rate of any other profession i.e. 8%. On the behalf of this report, software engineers have a very bright future in the software engineering field in the upcoming years. Software Engineering is the systematic and scientific approach to develop, deploy and maintain software systems.

As we know, requirements elicitation, design, testing, coding, and maintenance are the main phases of traditional software development. The traditional software development process is time-consuming and expensive. Moreover, instead of writing the code manually, data is used to program the system automatically. In addition to this, it is very difficult to make the machine intelligent with the help of software engineering. Nowadays, there are many automatic tools and techniques to accomplish each phase of software development. For example, automatic test case generation, automatic model creation from requirements, and predicting defects before implementation of software. Therefore, to simulate human intelligence in machines, AI is introduced in the field of SE.

Artificial Intelligence (AI) is a branch of computer science that focuses on making machines intelligent. All the traits of AI have not been captured [1]. Machine Learning (ML) is a branch of AI that provides innovations to companies for the development of Software. Software Companies such as Facebook [2] are now increasing the use of ML techniques while developing applications. SE and AI progress in their respective disciplines without communicating the research results to each other. Therefore, there should be an interaction between these two fields to achieve better results. Although, each discipline has its advantages or disadvantages, however, both disciplines deal with the development of software. The main objective of AI is to make the software behave intelligently whereas SE aims to build high-quality software with minimized cost and time.

Nowadays, the integration of AI/ML with software engineering is in the boom [3] [4]. Recent advances in software engineering make the AI techniques or models easy to use with the help of existing libraries or APIs [5]. For the development of software, software engineers are using various AI techniques or ML models to increase the efficiency of the system. For example, the application of ML models in the Software Fault Prediction domain predicts the faults before the testing phase starts [6]. Therefore, developers only focus on that part of code where faults can occur in the future which prevents failure in the later stages of software development. As a consequence, time, cost, and effort are reduced for developing a software system.

A. Emerging Research Areas

Integration of AI and SE gives rise to many new research areas as shown in Figure 1. Some disciplines that come into the light are as follows.

1. Ambient Intelligence- It is a developing technology that makes our environment intelligent i.e. sensitive and responsive to a human being. An environment of Smart Home enriched with temperature handling devices (radiators) and electro domestics (refrigerator) is a typical example of ambient intelligence [7].
2. Software Agents-These are small intelligent systems that depend on a knowledge base to reach a final decision. Decisions are based upon the experiences of the experts and they have the characteristic of working in the background and giving results while triggering the situation. An antivirus program is the best example of a software agent which remains active in the background and give an alert while finding a new virus [8]
3. Computational Intelligence- It comprises adaptive concepts and methods that exhibit intelligent

979-8-3503-0071-0/23/$31.00 ©2023 IEEE

behavior in complex and dynamic environments. It helps to find solutions that are robust and approximative at the same time. Some computational intelligence algorithms have been used to find breast cancer using prostate cancer predictive modeling.

4. Knowledge-Based systems: Knowledge-based systems were earlier expert systems and the heart of developing any expert system. These are the computer programs that rely upon artificial intelligence and uses expert knowledge and experiences to perform tasks. A Knowledge-based system is designed to diagnose various neck diseases [9].

II. Related Work

Rech and Althoff [10] focused on different aspects of software engineering and artificial intelligence. They also explored new research areas where a combination of AI and SE work together. An overview of Agent-Oriented software engineering, Knowledge-based software engineering, Computational Intelligence, and ambient Intelligence is also discussed by the authors. From this paper, it is clear that there is a strong association between SE and AI that motivates researchers to do further research. Yin et al. proposed a requirement engineering model using The Open Innovation in Requirements Engineering (OIRE) method. They also used Logistic regression techniques to know whether the lock is assigned to the repository or not and the MatGap tool for analysis of business rules [11].

Another paper by Tian et al. deals with the coding phase of software engineering. They proposed a model which automatically transfers natural language queries to API usage patterns with API-McTree decoder. Clustering and neural network are used in this paper [12]. On the other hand, Kim et al. [13] proposed the model Word2Vec for the generation of semantic features and then used machine learning classifiers to automatically validate the log levels in java irrespective of syntactic and semantic features. They used K-Nearest Neighbor and Random forest classifiers for validation of log levels and parameters are tuned using sensitivity analysis.

Feldt et al. [5] present a taxonomy of application Artificial Intelligence on Software Engineering levels. The motivation of providing this taxonomy is to be aware of the advantages and disadvantages of applying artificial intelligence in the software engineering domain. It will help the researchers to find a suitable technique while developing a software system. They proposed three facets- Type of AI applied, Point of Application, and level of automation offered. In addition to this, they analyze risk factors associated with applying AI techniques in the software engineering domain. Jain [1] presented a review that presents the interaction between AI and SE. The main objective of this paper is to apply tools and techniques of AI in SE or vice-versa in such a way that the benefits of these disciplines add together for helping society in real-world applications. In addition to this, the authors discussed frameworks, factors, and new research areas of interaction between two disciplines.

III. Challenges

During the fusion of AI/ML and SE, some challenges [14] are identified which is divided into the following three categories:

1. Development Challenges- There are fundamental differences for developing an ML-based system over a traditional software system, some of the challenges are discussed below:
   - Experimental challenges-For developing any software, using ML models, there is a need of repeating the experiments to find the optimal one. To produce repeatable results, users must know about the configuration (such as Hardware, Platform, Source code) of the system upon which it is implementing the model. A small variation in the configurational requirements may vary the results of the model. Secondly, Version Control of AI/Ml Systems generates different versions of data that will give different results during implementation. It is very difficult to keep track of the Data component and the cost of saving version data is very high [15]. Thirdly, different versions of ML models are created by varying the parameters' values and metrics, it is very difficult to manage models in long run. To perform hyperparameter tuning of models using optimization techniques different versions of the same data with different configuration parameters [16].

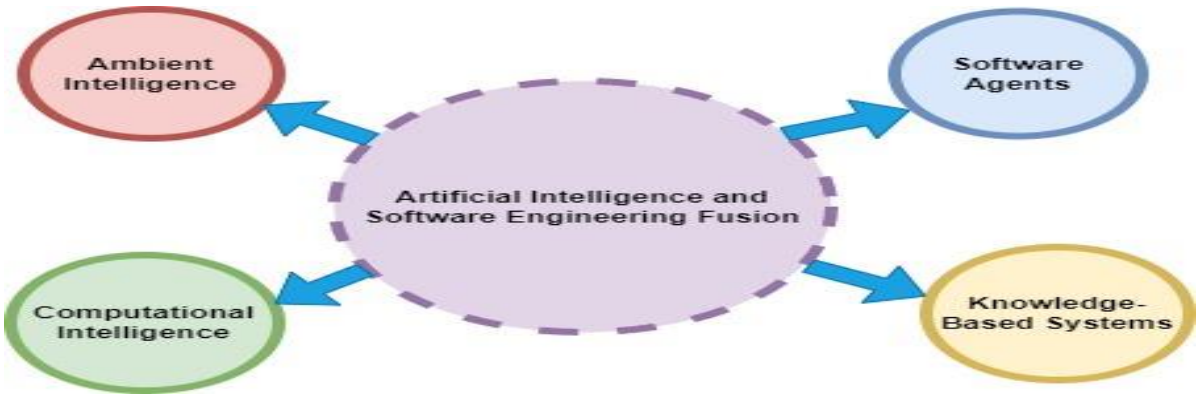

Fig.1. New Emerging areas of AI and SE fusion

   - Lack of transparency- In SE, a complex system is divided into multiple blocks which are further combined based on similar properties into different abstraction levels. However, in ML this

process is done automatically, so, it is difficult to identify the abstraction layers. Moreover, semantic information about the model is a typical task that is usually done by approximation methods [17].

- Difficult to troubleshoot- For the Development of Deep Learning systems, it is a challenging process to approximate the results before the system is undergoing the training and testing phase. Besides, it is very difficult to debug neural networks using traditional processes because we are not aware of the internal working of this. Another challenging issue is the integration of libraries in big data platforms makes it difficult to troubleshoot due to Lazy graph execution or lower adoption rate in the AI area. Moreover, training a neural with a large number of layers does not guarantee to achieve high performance always, so, a lot of effort is going to be wasted [10].
- Resource requirement- when dealing with distributed systems, a substantial number of resources are required. For example- the computational need for extracting or transforming data, training, and evaluation of model or serving the model n production. When working with distributed systems, data processing or training not only adds complexity, additional knowledge, and time to operate them.
- Testing- From the full dataset, choosing a sample consisting of all edge cases is a challenging problem. Moreover, training algorithms having non-deterministic nature makes the testing of models is a critical issue. There is a difference in production mode and implementation in terms of data processing and model serving compared to training or testing where training skews the model performance [10]. It became crucial to have a proper test in place.

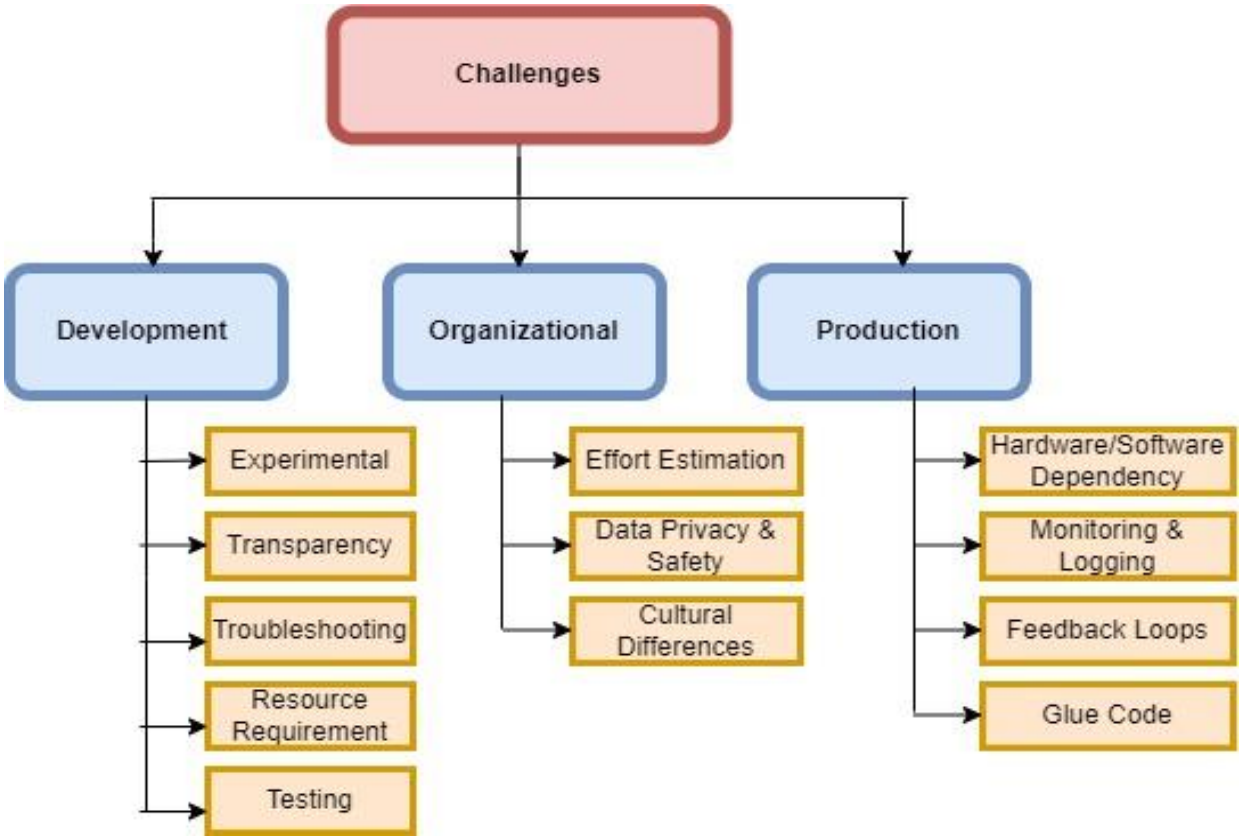

Fig.2. Challenges

2. Organizational Challenges- Developing an ML model requires collaboration with different teams with new ideas, priorities, and cultural values. Every team put an adequate amount of effort into the development of the software system.
    - Difficult to estimate Effort- We can easily estimate the time and resources required for the completion of a traditional software project. In AI/ML projects, it is unclear up to which extent a model can achieve its goal. Due to a lack of transparency, it is very hard to understand the working of the model and the updating the model to achieve better results.
    - Data privacy and Safety- As the internal working of the neural network is not known to the users, it has serious implications for privacy and data safety. Due to this, designers do not know where and how information is stored. Some companies prevent the usage of raw data as input to the model, instead, focus on aggregated statistics of user data that leads to difficult data exploration and troubleshooting problems. Some organizations such as European General Data Protection Regulation [18] works to keep data safe and protect privacy concerns. A lot of studies [10] worked upon the safety and privacy of datasets but still, more work is needed to preserve the safety and privacy of sensitive datasets.
    - Cultural Differences- Implementing a system requires collaboration among different people with different roles. In a team, a programmer focus on writing a code whereas other members focus on maintainability and quality of software. Both may be have cultural difference and different ways to do their assigned task which leads to difficulty in working under a team of developing any software project [19].
3. Production Challenges- It is important to use the latest hardware for the delivery of any software system. This yields a significant challenge of maintaining the latest hardware and their associated dependencies such as a change in the source of the dataset. Some challenges related to the production field are as follows:
    - Hardware/Software Dependency- For training DL systems, GPUs are used for improving performance. Increasing hardware performance leads to reduced training time, so, updation of new hardware provides an incentive for the software. Changing the hardware or software not only increases the cost but it is difficult to obtain reproducible results.
    - Monitoring and Logging - The effort required to maintain the deployed AI/ML system with time is difficult to predict by users [10]. As external behavior changes, the behavior of the ML model also changes. In this scenario, unit tests and integration tests are valuable but not sufficient for validation the system. Live performance monitoring is required but choosing metrics is a challenging task.
    - Unwanted Feedback Loops- As AI/ML systems are designed based on external data, their performance also depends upon external data, no matter how long the training time is. When models are deployed in a big data context, it suffers from an unintended feedback loop. These positive feedback loops are inherently unstable.
    - Glue code- AI/ML systems have the property that only 5% of the code deals with the system and the other glue code is used for integrating libraries and systems and also for supporting systems [20]. Updation of glue code and external changes in cloud services give rise to unexpected challenges in production-ready systems.

IV. Conclusion

In recent years, we conclude that the software industry is more inclined to develop AI-based systems. However, it is very challenging to create a production-ready system if there is a small research group and support system. It is also found that the fusion of SE and AI emerges in many new innovative areas which require enhanced research in the future. This paper also identifies various challenges while developing AI systems. Although the fusion of ML and SE gives promising results, still this area demands more research to handle the challenges. Solutions to these challenges are not only beneficial for researchers and software engineers, instead, a large number of companies around the globe benefits from the solutions.

References

[1] P. Jain, “Interaction between Software Engineering and Artificial Intelligence- A Review,” International Journal on Computer Science and Engineering (IJCSE), vol. 3, no. 12, pp. 3774–3779, 2011.

[2] K. Hazelwood et al., “Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective,” in Proceedings - International Symposium on High-Performance Computer Architecture, 2018, vol. 2018-Febru, pp. 620–629, doi: 10.1109/HPCA.2018.00059.

[3] S. Martínez-Fernández et al., “Software Engineering for AI-Based Systems: A Survey,” ACM Transactions on Software Engineering and Methodology, vol. 31, no. 2, pp. 1–58, 2022, doi: 10.1145/3487043.

[4] V. Vakkuri, K. K. Kemell, J. Tolvanen, M. Jantunen, E. Halme, and P. Abrahamsson, “How Do Software Companies Deal with Artificial Intelligence Ethics? A Gap Analysis,” ACM International Conference Proceeding Series, pp. 100–109, 2022, doi: 10.1145/3530019.3530030.

[5] R. Feldt, F. G. De Oliveira Neto, and R. Torkar, “Ways of applying artificial intelligence in software engineering,” Proceedings - International Conference on Software Engineering, pp. 35–41, 2018, doi: 10.1145/3194104.3194109.

[6] R. Malhotra, “A systematic review of machine learning techniques for software fault prediction,” Applied Soft Computing Journal, 2015, doi: 10.1016/j.asoc.2014.11.023.

[7] D. J. Cook, J. C. Augusto, and V. R. Jakkula, “Ambient intelligence: Technologies, applications, and opportunities,” Pervasive and Mobile Computing, vol. 5, no. 4, pp. 277–298, 2009, doi: 10.1016/j.pmcj.2009.04.001.

[8] A. Sołtysik-Piorunkiewicz, M. Furmankiewicz, and P. Ziuziański, "Artificial Intelligence Systems for Knowledge Management in e-Health: The Study of Intelligent Software Agents Technological-Organization-Environment Model (TOE) for mHealth View project," no. July, 2014, [Online]. Available: https://www.researchgate.net/publication/271014570.

[9] S. S. A. Naser and S. H. Almursheidi, "A Knowledge Based System for Neck Pain Diagnosis," World Wide journal of multidisciplinary research and development, vol. 2, no. 4, pp. 12–18, 2016.

[10] J. Rech and K.-D. Althoff, "Artificial intelligence and software engineering: Status and future trends," Themenschwerpunkt KI & SE, KI, vol. 3, no. June 2014, pp. 5–11, 2004.

[11] K. Yin, W. Chen, J. Zhou, G. Wu, and J. Wei, "STAR: A Specialized Tagging Approach for Docker Repositories," Proceedings - Asia-Pacific Software Engineering Conference, APSEC, vol. 2018-Decem, pp. 426–435, 2018, doi: 10.1109/APSEC.2018.00057.

[12] Y. Tian, X. Wang, H. Sun, Y. Zhao, C. Guo, and X. Liu, "Automatically Generating API Usage Patterns from Natural Language Queries," Proceedings - Asia-Pacific Software Engineering Conference, APSEC, vol. 2018-Decem, pp. 59–68, 2018, doi: 10.1109/APSEC.2018.00020.

[13] T. Kim, S. Kim, C. J. Yoo, S. Cho, and S. Park, "An Automatic Approach to Validating Log Levels in Java," Proceedings - Asia-Pacific Software Engineering Conference, APSEC, vol. 2018-Decem, pp. 623–627, 2018, doi: 10.1109/APSEC.2018.00078.

[14] A. Arpteg, P. Ab, P. Ab, and J. Bosch, "Software Engineering Challenges of Deep Learning," 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 50–59, 2018, doi: 10.1109/SEAA.2018.00018.

[15] J. D. Morgenthaler, M. Gridnev, R. Sauciuc, and S. Bhansali, "Searching for build debt: Experiences managing technical debt at Google," in 2012 3rd International Workshop on Managing Technical Debt, MTD 2012 - Proceedings, 2012, pp. 1–6, doi: 10.1109/MTD.2012.6225994.

[16] D. Golovin, B. Solnik, S. Moitra, G. Kochanski, J. Karro, and D. Sculley, "Google Vizier : A Service for Black-Box Optimization," in Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2017, pp. 1487–1495, doi: https://doi.org/10.1145/3097983.3098043.

[17] Y. Bengio, "Deep Learning of Representations for Unsupervised and Transfer Learning," JMLR: Workshop and Conference Proceedings, vol. 7, pp. 1–20, 2011.

[18] P. J. Hustinx, "EU Data Protection Law: The Review of Directive 95/46/EC and the Proposed General Data Protection Regulation"," Collected Courses of the European University Institute's Academy of European Law, vol. 24, no. January 2013, pp. 1–52, 2013, [Online]. Available: https://secure.edps.europa.eu/EDPSWEB/edps/EDPS/Publications/SpeechArticle/SA2014.

[19] S. Tata et al., "Quick access: Building a smart experience for google drive," in Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2017, vol. Part F1296, pp. 1643–1651, doi: 10.1145/3097983.3098048.

[20] D. Sculley et al., "Hidden technical debt in machine learning systems," in Advances in Neural Information Processing Systems, 2015, vol. 2015-Janua, pp. 2503–2511.